

Kubernetes-08

Tasks:

1. **Create and Test a Kubernetes Pod with an EmptyDir Volume**
2. **Configure a HostPath Volume in Kubernetes and Validate Data Persistence**
3. **Deploy an Amazon EBS Volume Using Persistent Volume and Persistent Volume Claim (PVC)**
4. **Set Up an Amazon EFS Volume and Attach it to Multiple Pods**
5. **Implement and Test Liveness and Readiness Probes in a Kubernetes Pod**

Task:

1. Create and Test a Kubernetes Pod with an EmptyDir Volume

We will:

1. Create Pod with EmptyDir
2. Create files
3. Restart container → verify data exists
4. Delete pod → verify data lost

Step 1: Create EmptyDir Pod

Create file:

```
vi emptydir-pod.yaml
```

A terminal window with a black background and green text. It shows the command 'vi emptydir-pod.yaml' being executed, followed by 'cat emptydir-pod.yaml' which displays the contents of the file. The file is a Kubernetes Pod manifest for a container named 'nginx' with an 'nginx' image. It includes a volume named 'myvolume' of type 'EmptyDir' mounted at '/data'.

```
ubuntu@master:~$ vi emptydir-pod.yaml
[ubuntu@master:~$ cat emptydir-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: myvolume
      mountPath: /data
  volumes:
  - name: myvolume
    emptyDir: {}

ubuntu@master:~$ █
```

Apply it:

```
kubectl apply -f emptydir-pod.yaml
```

Check:

```
kubectl get pods
```

Wait until STATUS = Running.

```
ubuntu@master:~$ kubectl apply -f emptydir-pod.yaml
pod/emptydir-pod created
ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
emptydir-pod  1/1     Running   0           6s
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
emptydir-pod  1/1     Running   0          14s   192.168.235.131 worker1    <none>           <none>
```

Step 2: Create Test Files

Enter the pod:

```
kubectl exec -it emptydir-pod -- bash
Inside container:
```

```
cd /data
```

```
touch tiger1 tiger2 tiger3
```

```
ls
```

```
emptydir-pod  1/1     Running   0          14s   192.168.235.131
ubuntu@master:~$ kubectl exec -it emptydir-pod -- bash
[root@emptydir-pod:/# cd /data
[root@emptydir-pod:/data# touch tiger1 tiger2 tiger3
[root@emptydir-pod:/data# ls
tiger1 tiger2 tiger3
root@emptydir-pod:/data#
```

Step 3: Restart Container (Not Pod)

Now stop nginx process:

```
kubectl exec -it emptydir-pod -- service nginx stop
```

Check:

```
kubectl get pods
```

```
Command terminated with exit code 127
ubuntu@master:~$ kubectl exec -it emptydir-pod -- service nginx stop
command terminated with exit code 137
ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
emptydir-pod  1/1     Running   1 (6s ago)  3m14s
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
emptydir-pod  1/1     Running   1 (10s ago)  3m18s  192.168.235.131  worker1    <none>           <none>
```

You will see:

RESTARTS = 1

Now check files again:

```
kubectl exec -it emptydir-pod -- ls /data
```

```
emptydir-pod  1/1     Running   1 (10s ago)  3m18s  192.168.235.131  v
ubuntu@master:~$ kubectl exec -it emptydir-pod -- ls /data
tiger1 tiger2 tiger3
ubuntu@master:~$
```

Files will still be there.

- ✓ Because EmptyDir survives container restart
- ✓ Pod is still same

Step 4: Delete Pod (optional just to show it does not save data if the pod is deleted)

Now delete pod:

```
kubectl delete pod emptydir-pod
```

Recreate:

```
kubectl apply -f emptydir-pod.yaml
```

Now check again:

```
kubectl exec -it emptydir-pod -- ls /data
```

✗ Folder will be empty.

```
ubuntu@master:~$ kubectl delete pod emptydir-pod
pod "emptydir-pod" deleted
ubuntu@master:~$ kubectl apply -f emptydir-pod.yaml
pod/emptydir-pod created
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
emptydir-pod   1/1     Running   0          10s   192.168.235.134 worker1      <none>           <none>
ubuntu@master:~$ kubectl exec -it emptydir-pod -- ls /data
ubuntu@master:~$
```

Final Conclusion

Action	Data Status
Container Restart	✓ Data stays
Pod Delete	✗ Data lost

Because:

EmptyDir lifetime = Pod lifetime

Task:2

2.Configure a HostPath Volume in Kubernetes and Validate Data Persistence

This one is very important for understanding **node-level storage**.

Goal

- ✓ Create a Pod using HostPath
- ✓ Create files
- ✓ Delete Pod
- ✓ Recreate Pod
- ✓ Verify data still exists

First Understand HostPath

- HostPath stores data on **Node machine**
- Not inside container
- Not inside Pod

- Directly on Node filesystem

So:

Container restart → Data stays ✅

Pod delete → Data stays ✅

But if Pod moves to another node → Data NOT available ❌

STEP 1: Create HostPath Pod

Create YAML:

```
vi hostpath-pod.yaml
```

```
pod -mpyall pod -delete
ubuntu@master:~$ vi hostpath-pod.yaml
[ubuntu@master:~$ cat hostpath-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: host-volume
      mountPath: /data
  volumes:
  - name: host-volume
    hostPath:
      path: /mnt/hostdata
      type: DirectoryOrCreate

ubuntu@master:~$ █
```

Save and exit.

Apply it:

```
kubectl apply -f hostpath-pod.yaml
```

Check:

```
kubectl get pods
```

Wait until Running.

```
ubuntu@master:~$ kubectl apply -f hostpath-pod.yaml
pod/hostpath-pod created
ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
hostpath-pod  1/1     Running   0           16s
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
hostpath-pod  1/1     Running   0           21s   192.168.235.135  worker1    <none>            <none>
```

STEP 2: Create Test File

Enter pod:

```
kubectl exec -it hostpath-pod -- bash
```

Inside container:

```
cd /data
touch hostfile1 hostfile2
ls
```

You should see files.


```
hostpath-pod 1/1 Running 0 21s 192.168.23
ubuntu@master:~$ kubectl exec -it hostpath-pod -- bash

[root@hostpath-pod:/# cd /data
[root@hostpath-pod:/data# touch hostfile1 hostfile2
[root@hostpath-pod:/data# ls
hostfile1 hostfile2
root@hostpath-pod:/data#
```

STEP 3: Verify on Node (Very Important)

Now check directly on node machine:

```
ls /mnt/hostdata
You will see:
```

```
hostfile1 hostfile2
```

This proves data is stored on Node.

```
ubuntu@worker1:~$ ls
ubuntu@worker1:~$ cd /mnt/hostdata/
ubuntu@worker1:/mnt/hostdata$ ls
hostfile1 hostfile2
ubuntu@worker1:/mnt/hostdata$
```

STEP 4: Delete Pod

```
kubectl delete pod hostpath-pod  
Recreate:
```

```
kubectl apply -f hostpath-pod.yaml  
Now check again:
```

```
kubectl exec -it hostpath-pod -- ls /data
```

```
ubuntu@master:~$ kubectl delete pod hostpath-pod  
pod "hostpath-pod" deleted  
ubuntu@master:~$ kubectl apply -f hostpath-pod.yaml  
pod/hostpath-pod created  
ubuntu@master:~$ kubectl get pods  
NAME          READY   STATUS    RESTARTS   AGE  
hostpath-pod   1/1     Running   0           41s  
ubuntu@master:~$ kubectl get pods -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES  
hostpath-pod   1/1     Running   0           69s   192.168.235.137 worker1      <none>           <none>  
ubuntu@master:~$ kubectl exec -it hostpath-pod -- ls /data  
hostfile1 hostfile2  
ubuntu@master:~$
```

Files still there.

Because data lives on Node.

Final Understanding

Storage Type	Container Restart	Pod Delete	Pod Moves Node
emptyDir	✓	✗	✗
hostPath	✓	✓	✗

Important Production Note

HostPath is:

- ✗ Not recommended for production
- ✗ Not safe in multi-node clusters
- ✓ Mostly for learning or single-node setups

For production → use PVC (EBS, EFS, etc.)

Task:3

3. Deploy an Amazon EBS Volume Using Persistent Volume and Persistent Volume Claim (PVC)

Flow (Static Provisioning)

1. Create EBS manually in AWS
2. Create PersistentVolume (PV)
3. Create PVC
4. Create Pod using PVC
5. Test persistence

STEP 1: Create EBS Volume Manually in AWS

Go to:

AWS Console → EC2 → Volumes → Create Volume

Choose:

- Type: gp3 (or gp2)
- Size: 2 GiB
- Availability Zone: MUST match your worker node AZ
- Leave rest default

Click Create.

Copy the Volume ID

Successfully created volume [vol-0e835da3a6073824f](#).

Volumes (1/5) [Info](#) Last updated less than a minute ago [Recycle Bin](#)

Saved filter sets: No managed volumes [Clear filters](#)

Managed = false ☐ [X](#)

☐	Name ↗	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume ID
☐		vol-043f1054f52770931	gp3	20 GiB	3000	125	snap-0b6c5b3...	-
☐		vol-05253131f9adac852	gp3	8 GiB	3000	125	snap-0fd1b5...	-
☐		vol-05ddd97b00d538cb2	gp3	20 GiB	3000	125	snap-0b6c5b3...	-
☒	my-vol ↗	vol-0e835da3a6073824f	gp3	20 GiB	3000	125	-	-
☐		vol-093bbf44d1d6cd5af	gp3	20 GiB	3000	125	snap-0b6c5b3...	-

Volume ID: [vol-0e835da3a6073824f](#) (my-vol)

[Details](#) [Status checks](#) [Monitoring](#) [Tags](#)

Volume ID vol-0e835da3a6073824f (my-vol)	Size 20 GiB	Type gp3	Status check 🟢 Okay
AWS Compute Optimizer finding ⓘ Opt-in to AWS Compute Optimizer for recommendations. Learn more	Volume state 🟢 Available	IOPS 3000	Throughput 125

Step 2: Create IAM Role for EBS CSI Driver

- Go to IAM → Roles → Create Role
- Select: AWS Service → EC2
- Attach Policy:

AmazonEBSCSIDriverPolicy

-

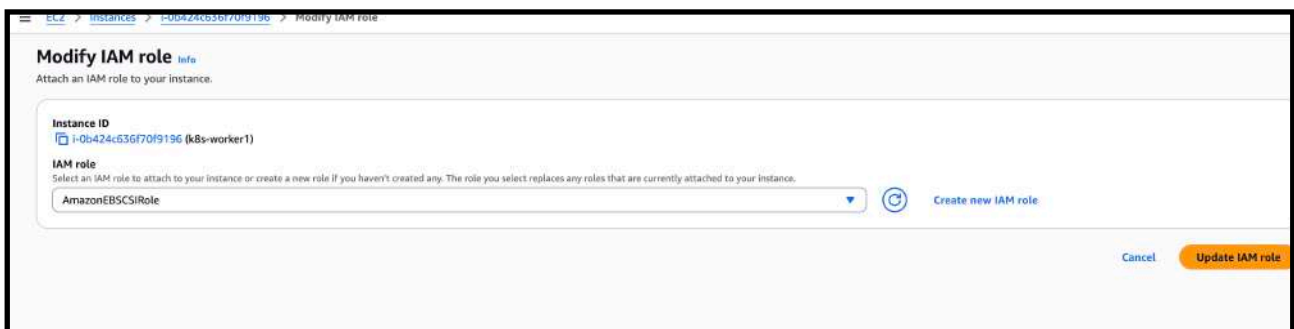
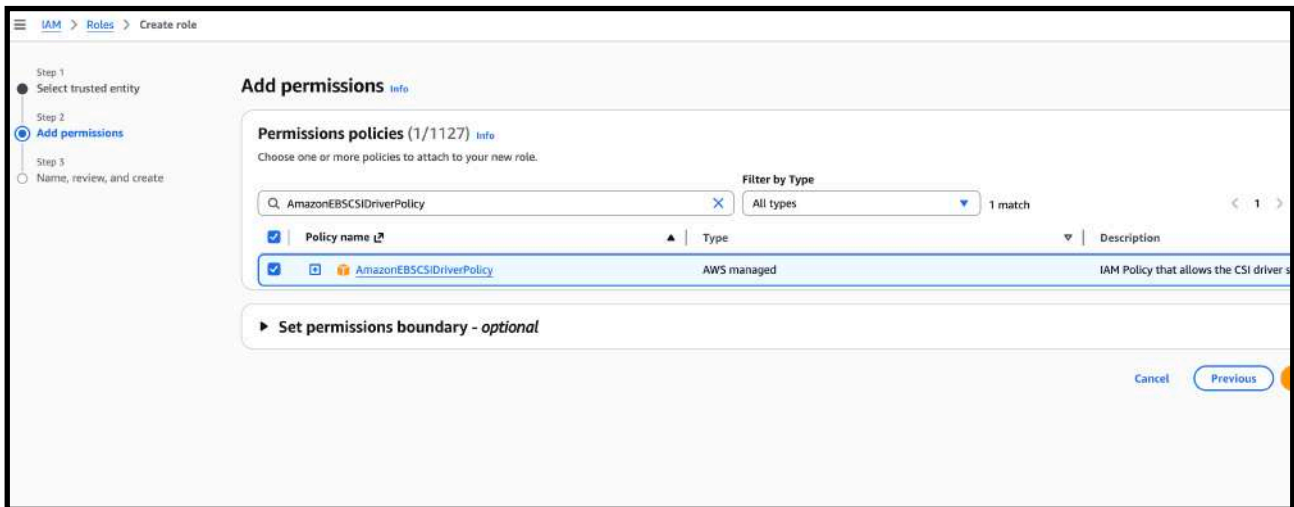
- Name the role:

AmazonEBSCSIRole

-

- Attach this role to:
 - k8s-worker1

- k8s-worker2



—> add to other worker node also

Step 3: Install AWS EBS CSI Driver

On master node:

```
git clone https://github.com/kubernetes-sigs/aws-ebs-csi-
driver.git
cd aws-ebs-csi-driver
kubectl apply -k deploy/kubernetes/overlays/stable/
Verify installation:
```

```
kubectl get pods -n kube-system
```

```
Last login: Wed Feb 25 10:25:20 2026 from 49.43.224.29
ubuntu@master:~$ git clone https://github.com/kubernetes-sigs/aws-ebs-csi-driver.git
cd aws-ebs-csi-driver
kubectl apply -k deploy/kubernetes/overlays/stable/
Cloning into 'aws-ebs-csi-driver'...
remote: Enumerating objects: 37437, done.
remote: Counting objects: 100% (979/979), done.
remote: Compressing objects: 100% (282/282), done.
remote: Total 37437 (delta 814), reused 714 (delta 689), pack-reused 36458 (from 2)
Receiving objects: 100% (37437/37437), 31.25 MiB | 33.68 MiB/s, done.
Resolving deltas: 100% (21679/21679), done.
serviceaccount/ebs-csi-controller-sa created
serviceaccount/ebs-csi-node-sa created
role.rbac.authorization.k8s.io/ebs-csi-leases-role created
clusterrole.rbac.authorization.k8s.io/ebs-csi-node-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-attacher-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-provisioner-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-resizer-role created
clusterrole.rbac.authorization.k8s.io/ebs-external-snapshotter-role created
rolebinding.rbac.authorization.k8s.io/ebs-csi-leases-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-attacher-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-node-getter-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-provisioner-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-resizer-binding created
clusterrolebinding.rbac.authorization.k8s.io/ebs-csi-snapshotter-binding created
deployment.apps/ebs-csi-controller created
poddisruptionbudget.policy/ebs-csi-controller created
daemonset.apps/ebs-csi-node created
Error from server (BadRequest): error when creating "deploy/kubernetes/overlays/stable/": CSIDriver in version "v1" cannot be h
```

```
kube-scheduler-master 1/1 Running 8 (100m ago) 8d
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
calico-kube-controllers-5b9b456c66-m9q5l 1/1     Running   8 (100m ago) 8d
calico-node-5vzw9                1/1     Running   8 (100m ago) 8d
calico-node-nrkps                 1/1     Running   8 (100m ago) 8d
calico-node-xvrb2                 1/1     Running   8 (100m ago) 8d
coredns-55cb58b774-2xs8s          1/1     Running   8 (100m ago) 8d
coredns-55cb58b774-57jgs          1/1     Running   8 (100m ago) 8d
ebs-csi-controller-55cbf6bcd4-ndqhh 6/6     Running   0           32s
ebs-csi-controller-55cbf6bcd4-nfjtx 6/6     Running   0           32s
ebs-csi-node-6vdjq                3/3     Running   0           32s
ebs-csi-node-9rkkc                3/3     Running   0           32s
ebs-csi-node-fjjc7                3/3     Running   0           32s
etcd-master                       1/1     Running   8 (100m ago) 8d
kube-apiserver-master             1/1     Running   8 (100m ago) 8d
kube-controller-manager-master    1/1     Running   8 (100m ago) 8d
kube-proxy-9m9hd                  1/1     Running   8 (100m ago) 8d
kube-proxy-jvw5b                  1/1     Running   8 (100m ago) 8d
kube-proxy-mj4x2                  1/1     Running   8 (100m ago) 8d
kube-scheduler-master             1/1     Running   8 (100m ago) 8d
ubuntu@master:~/aws-ebs-csi-driver$
```

Ensure:

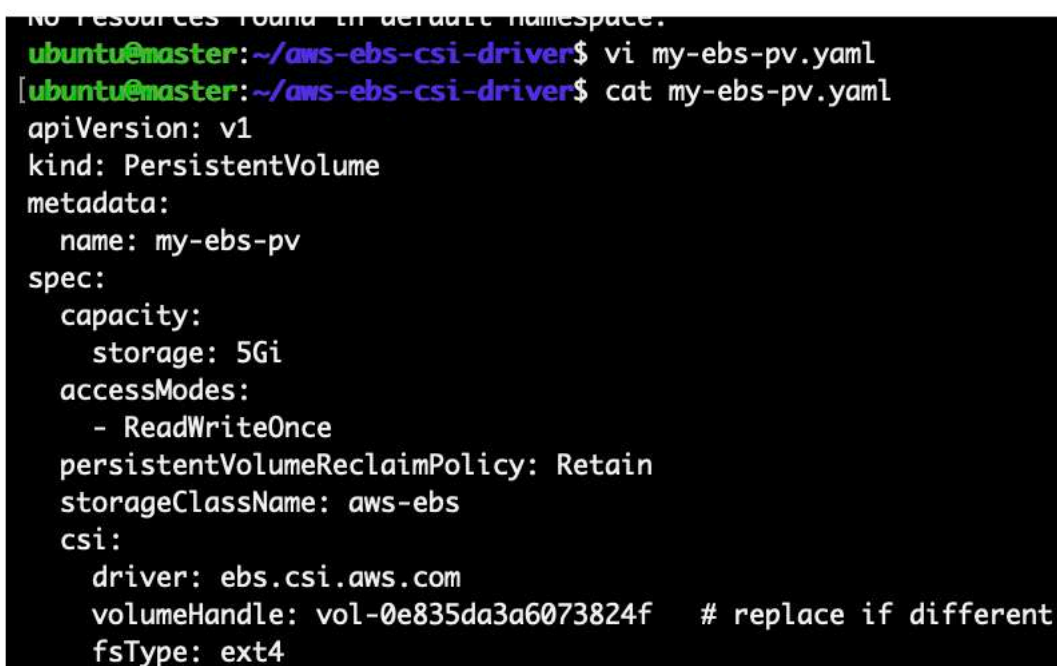
- ebs-csi-controller → Running

- ebs-csi-node → Running

Step 4 — Create Persistent Volume (Static CSI)

Create file:

```
vi my-ebs-pv.yaml
```



```
No resources found in default namespace.  
ubuntu@master:~/aws-ebs-csi-driver$ vi my-ebs-pv.yaml  
[ubuntu@master:~/aws-ebs-csi-driver$ cat my-ebs-pv.yaml  
apiVersion: v1  
kind: PersistentVolume  
metadata:  
  name: my-ebs-pv  
spec:  
  capacity:  
    storage: 5Gi  
  accessModes:  
    - ReadWriteOnce  
  persistentVolumeReclaimPolicy: Retain  
  storageClassName: aws-ebs  
  csi:  
    driver: ebs.csi.aws.com  
    volumeHandle: vol-0e835da3a6073824f    # replace if different  
    fsType: ext4
```

Apply:

```
kubectl apply -f my-ebs-pv.yaml
```

Check:

```
kubectl get pv
```



```
ubuntu@master:~/aws-ebs-csi-driver$ kubectl apply -f my-ebs-pv.yaml
persistentvolume/my-ebs-pv created
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM          STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
my-ebs-pv     5Gi       RWO           Retain          Available  my-ebs-pv     aws-ebs       <unset>               4s
```

It should show:

STATUS: Available

Step 5 — Create PersistentVolumeClaim

Create file:

```
vi my-ebs-pvc.yaml
```

```
ubuntu@master:~/aws-ebs-csi-driver$ vi my-ebs-pvc.yaml
[ubuntu@master:~/aws-ebs-csi-driver$ at my-ebs-pvc.yaml
Command 'at' not found, but can be installed with:
sudo apt install at
[ubuntu@master:~/aws-ebs-csi-driver$ cat my-ebs-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-ebs-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: aws-ebs
  resources:
    requests:
      storage: 5Gi
```

Save and apply:

```
kubectl apply -f my-ebs-pvc.yaml
```

Now check:


```
kubectl get pvc
kubectl get pv
```

```
ubuntu@master:~/aws-ebs-csi-driver$ kubectl apply -f my-ebs-pvc.yaml
persistentvolumeclaim/my-ebs-pvc created
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pvc
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
my-ebs-pvc    Pending             5Gi        RWO                aws-ebs                <unset>                 5s
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pv
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
my-ebs-pv     Pending             5Gi        RWO                aws-ebs                <unset>                 24s
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pv
NAME          CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM   STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
my-ebs-pv     5Gi        RWO                Retain            Available            aws-ebs                <unset>                 76s
```

Next Step — Create Pod

Create file:

```
vi my-pod.yaml
```

```
ubuntu@master:~/aws-ebs-csi-driver$ vi my-pod.yaml
ubuntu@master:~/aws-ebs-csi-driver$ cat my-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: data-volume
      mountPath: /data
  volumes:
  - name: data-volume
    persistentVolumeClaim:
      claimName: my-ebs-pvc
ubuntu@master:~/aws-ebs-csi-driver$ kubectl apply -f my-pod.yaml
pod/my-pod created
```

Apply:

```
kubectl apply -f my-pod.yaml
```

Now check:

```
kubectl get pvc
```

```
kubectl get pv
```

```
kubectl get pods
```

Now PVC should move to **Bound**

PV should move to **Bound**

Pod should move to **Running**

```
ubuntu@master:~/aws-ebs-csi-driver$ kubectl apply -f my-pod.yaml
pod/my-pod created
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pvc
kubectl get pv
kubectl get pods
NAME          STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
my-ebs-pvc    Bound  my-ebs-pv  5Gi      RWO           aws-ebs      <unset>                2m14s
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM          STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
my-ebs-pv     5Gi      RWO           Retain          Bound   default/my-ebs-pvc  aws-ebs      <unset>                3m6s
NAME          READY  STATUS  RESTARTS  AGE
my-pod        1/1    Running  0         6s
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM          STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
my-ebs-pv     5Gi      RWO           Retain          Bound   default/my-ebs-pvc  aws-ebs      <unset>                3m11s
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pvc
NAME          STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
my-ebs-pvc    Bound  my-ebs-pv  5Gi      RWO           aws-ebs      <unset>                2m24s
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pods
NAME          READY  STATUS  RESTARTS  AGE
my-pod        1/1    Running  0         20s
ubuntu@master:~/aws-ebs-csi-driver$
```

Testing Steps

Testing Step 1: Verify EBS Volume Attachment

- Login to AWS Console.
- Go to EC2 → Volumes.
- Verify the manually created EBS volume.
- Confirm:
 - Volume state is **in-use**.
 - Volume is attached to one of the worker nodes.

☒	my-vol	vol-0e835da3a6073824f	gp3	20 GiB	3000	125
☐		vol-093bbf44d1d6cd5af	gp3	20 GiB	3000	125

Volume ID: vol-0e835da3a6073824f (my-vol)

Details	Status checks	Monitoring	Tags
Volume ID vol-0e835da3a6073824f (my-vol)		Size 20 GiB	Type gp3
AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more		Volume state In-use	IOPS 3000
Fast snapshot restored		Availability Zone	Created

Testing Step 2: Create a Test File Inside the Pod

Execute:

```
kubectl exec -it my-pod -- bash
```

Inside the container:

```
cd /data
touch test-file
ls
exit
```

```
Last login: Wed Feb 23 10:41:42 2020 from 49.45.224.29
ubuntu@master:~$ kubectl exec -it my-pod -- bash
[root@my-pod:/# cd /data
[root@my-pod:/data# touch waseem-test
[root@my-pod:/data# ls
lost+found waseem-test
[root@my-pod:/data# exit
exit
```

Testing Step 3: Delete the Pod

```
kubectl delete pod my-pod
```

Wait until the pod is terminated.

Testing Step 4: Recreate the Pod

```
kubectl apply -f my-pod.yaml
```

Wait until the pod status becomes **Running**.

```
ubuntu@master:~$ kubectl delete pod my-pod
pod "my-pod" deleted
```

Testing Step 5: Verify Data Persistence

Execute:

```
kubectl exec -it my-pod -- ls /data
```

Verify that `test-file` is still present.

```
ubuntu@master:~/aws-ebs-csi-driver$ kubectl apply -f my-pod.yaml
pod/my-pod created
ubuntu@master:~/aws-ebs-csi-driver$ kubectl get pods
NAME      READY   STATUS             RESTARTS   AGE
my-pod    0/1     ContainerCreating   0          6s
[ubuntu@master:~/aws-ebs-csi-driver$
ubuntu@master:~/aws-ebs-csi-driver$ kubectl exec -it my-pod -- ls /data
lost+found  waseem-test
ubuntu@master:~/aws-ebs-csi-driver$
```

Testing Result

- The EBS volume successfully reattached to the worker node.
- The test file remained intact.
- This confirms that static provisioning using CSI driver is working correctly.
- Data persists even after pod deletion.

Task:4

4.Set Up an Amazon EFS Volume and Attach it to Multiple Pods

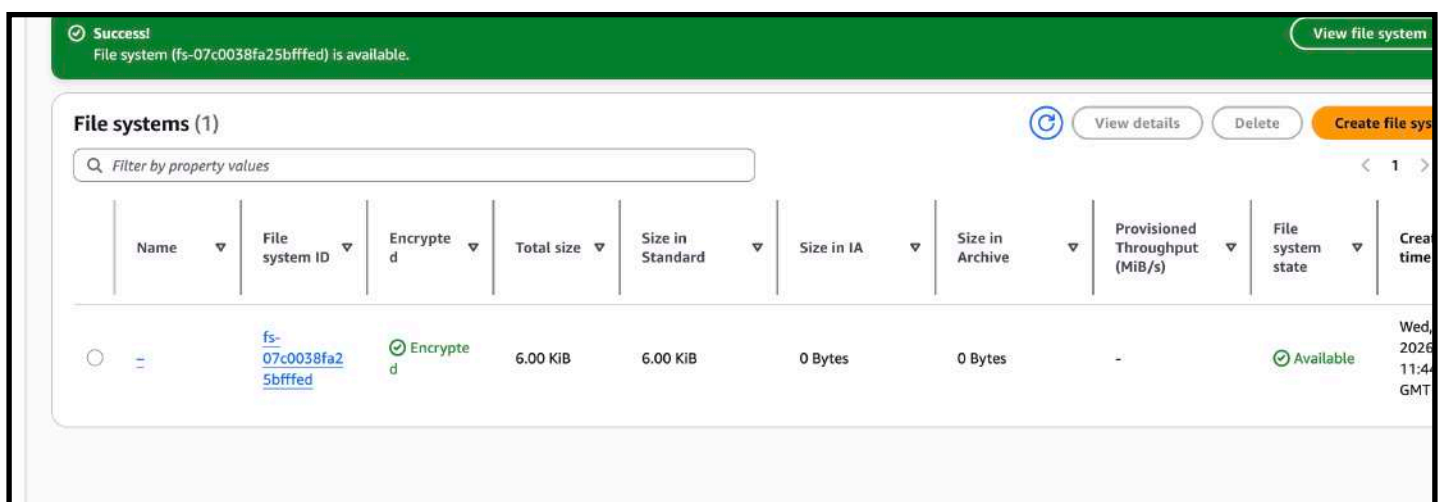
STEP 1 — Create EFS in AWS

Go to:

AWS → EFS → Create File System

Choose:

- Same VPC as EC2 nodes
- Enable mount targets for all subnets used by worker nodes
- Keep default settings



After Creation

1. Open the created EFS.
2. Copy the:

File System ID (fs-xxxxxxx)

Your File System ID is:

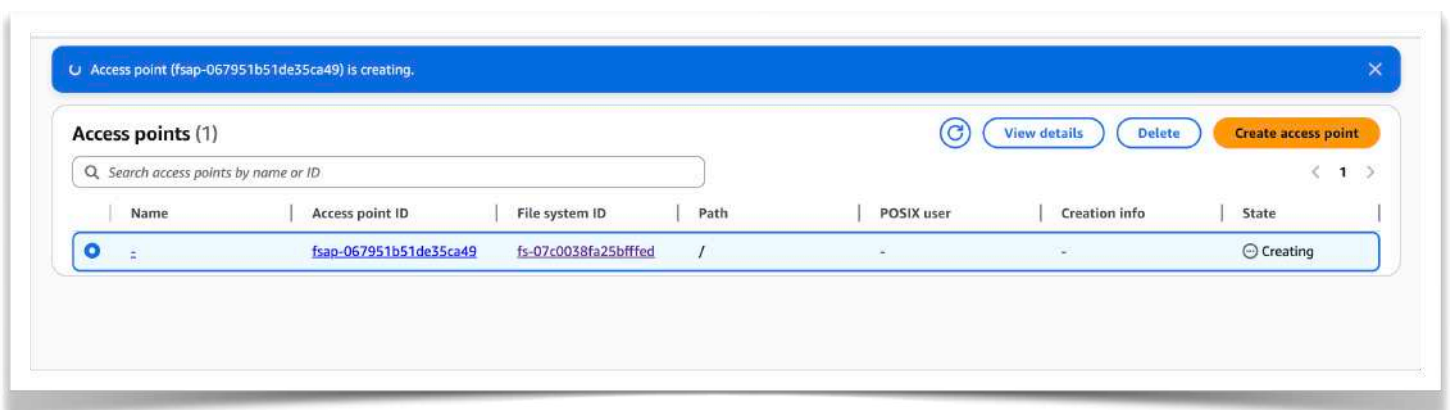
fs-07c0038fa25bffffed

Next Step — Create Access Point

Now do this:

1. Click **Access points** tab
2. Click **Create access point**
3. Keep everything default
4. Click **Create access point**

After creation, copy the:



Your Access Point ID is:

fsap-067951b51de35ca49

So now we have:

FileSystem ID → fs-07c0038fa25bffffed

AccessPoint ID → fsap-067951b51de35ca49

Next Step — Install EFS CSI Driver

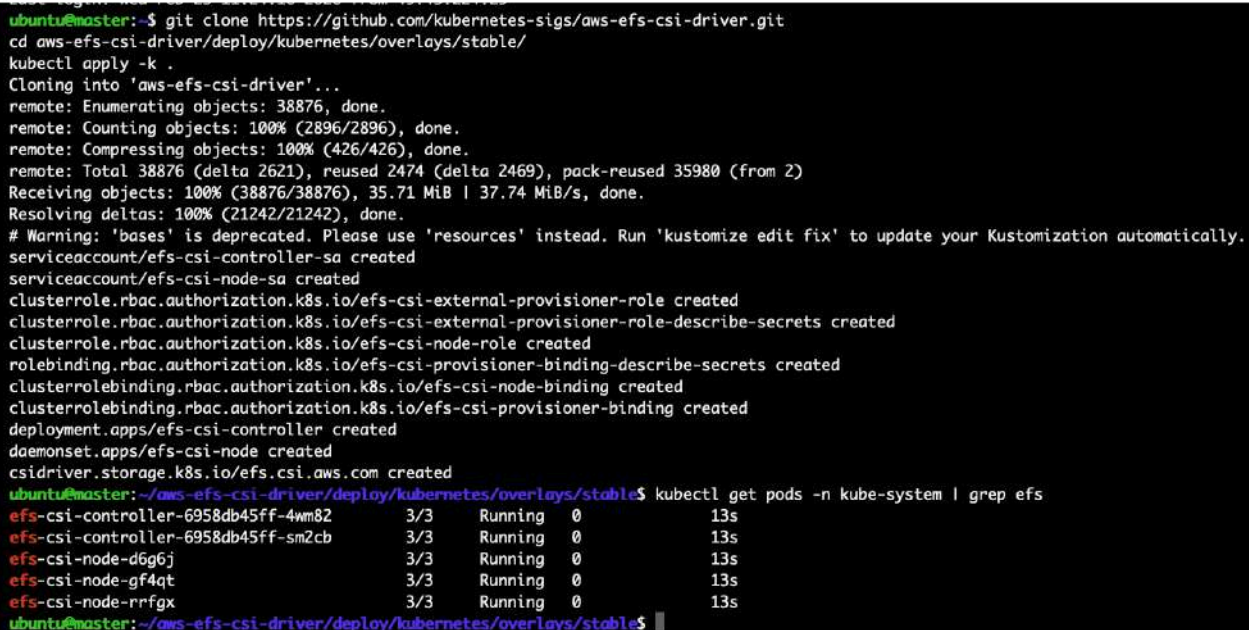
Now go to your master node.

Run:

```
git clone https://github.com/kubernetes-sigs/aws-efs-csi-driver.git
cd aws-efs-csi-driver/deploy/kubernetes/overlays/stable/
kubectl apply -k .
```

Then verify:

```
kubectl get pods -n kube-system | grep efs
```



```
ubuntu@master:~$ git clone https://github.com/kubernetes-sigs/aws-efs-csi-driver.git
cd aws-efs-csi-driver/deploy/kubernetes/overlays/stable/
kubectl apply -k .
Cloning into 'aws-efs-csi-driver'...
remote: Enumerating objects: 38876, done.
remote: Counting objects: 100% (2896/2896), done.
remote: Compressing objects: 100% (426/426), done.
remote: Total 38876 (delta 2621), reused 2474 (delta 2469), pack-reused 35980 (from 2)
Receiving objects: 100% (38876/38876), 35.71 MiB | 37.74 MiB/s, done.
Resolving deltas: 100% (21242/21242), done.
# Warning: 'bases' is deprecated. Please use 'resources' instead. Run 'kustomize edit fix' to update your Kustomization automatically.
serviceaccount/efs-csi-controller-sa created
serviceaccount/efs-csi-node-sa created
clusterrole.rbac.authorization.k8s.io/efs-csi-external-provisioner-role created
clusterrole.rbac.authorization.k8s.io/efs-csi-external-provisioner-role-describe-secrets created
clusterrole.rbac.authorization.k8s.io/efs-csi-node-role created
rolebinding.rbac.authorization.k8s.io/efs-csi-provisioner-binding-describe-secrets created
clusterrolebinding.rbac.authorization.k8s.io/efs-csi-node-binding created
clusterrolebinding.rbac.authorization.k8s.io/efs-csi-provisioner-binding created
deployment.apps/efs-csi-controller created
daemonset.apps/efs-csi-node created
csidriver.storage.k8s.io/efs.csi.aws.com created
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl get pods -n kube-system | grep efs
efs-csi-controller-6958db45ff-4wm82      3/3      Running    0          13s
efs-csi-controller-6958db45ff-sm2cb      3/3      Running    0          13s
efs-csi-node-d6g6j                       3/3      Running    0          13s
efs-csi-node-gf4qt                       3/3      Running    0          13s
efs-csi-node-rrfgx                       3/3      Running    0          13s
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$
```

All EFS CSI pods are **Running (3/3)**.

That means:

- ✓ EFS driver installed correctly
- ✓ IAM role is working
- ✓ Cluster can communicate with AWS
- ✓ No NFS port issue

You're doing very good now.

Next Step — Create StorageClass

You are currently inside:

```
~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable
```

That's fine. You can create YAML here.

Create file:

Create file:

```
vi storageclass.yaml
```

```
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ vi storageclass.yaml
[ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ cat storageclass.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: efs-sc
provisioner: efs.csi.aws.com

ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$
```

Now apply:


```
kubectl apply -f storageclass.yaml
```

Verify:

```
kubectl get sc
```

You must see:

```
efs-sc    efs.csi.aws.com
```

```
[ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl apply -f storageclass.yaml
storageclass.storage.k8s.io/efs-sc created
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl get sc
NAME      PROVISIONER      RECLAIMPOLICY   VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
aws-ebs   ebs.csi.aws.com   Delete          WaitForFirstConsumer false                   68m
efs-sc    efs.csi.aws.com   Delete          Immediate            false                   6s
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$
```

NEXT STEP — Create Persistent Volume (Static EFS)

Now create file:

```
vi efs-pv.yaml
```

```
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ vi efs-pv.yaml
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ cat efs-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: efs-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: efs-sc
  csi:
    driver: efs.csi.aws.com
    volumeHandle: fs-07c0038fa25bfffed::fsap-067951b51de35ca49
```

Save and apply:

```
kubectl apply -f efs-pv.yaml
```

Then check:

```
kubectl get pv
```

You should see:

```
efs-pv    Available
```

```
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl apply -f efs-pv.yaml
persistentvolume/efs-pv created
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
efs-pv	5Gi	RWX	Retain	Available		efs-sc	<unset>		5s

NEXT STEP — Create PVC for EFS

Now create:

```
vi efs-pvc.yaml
```

```
my-efs-pv    5Gi    RWX    Retain    Bound    default/my-efs-pvc    aws-efs
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ vi efs-pvc.yaml
[ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ cat efs-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: efs-pvc
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc
  resources:
    requests:
      storage: 5Gi

ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$
```

Save and apply:

```
kubectl apply -f efs-pvc.yaml
```

Now check:

```
kubectl get pvc
```

```
kubectl get pv
```

```
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl apply -f efs-pvc.yaml
persistentvolumeclaim/efs-pvc created
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
efs-pvc	Bound	efs-pv	5Gi	RWX	efs-sc	<unset>	13s

```
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
efs-pv	5Gi	RWX	Retain	Bound	default/efs-pvc	efs-sc	<unset>		2m38s

both show **Bound**

Now we create Pod 1 and Pod 2 to test RWX (multiple pods using same EFS).

STEP 1 — Create Pod 1

Create file:

```
vi efs-pod1.yaml
```

```

my-efs-pv 5Gi RWO Retain Bound default/my-efs-pvc aws-efs
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ vi efs-pod1.yaml
[ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ cat efs-pod1.yaml
apiVersion: v1
kind: Pod
metadata:
  name: efs-pod1
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: efs-volume
      mountPath: /efs-data
  volumes:
  - name: efs-volume
    persistentVolumeClaim:
      claimName: efs-pvc

ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$

```

Apply:

```
kubectl apply -f efs-pod1.yaml
```

```

claimName: efs-pvc
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl apply -f efs-pod1.yaml
pod/efs-pod1 created
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$

```

STEP 2 — Create Pod 2

```
vi efs-pod2.yaml
```

```

pod/efs-pod1 created
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ vi efs-pod2.yaml
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ cat efs-pod2.yaml
apiVersion: v1
kind: Pod
metadata:
  name: efs-pod2
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: efs-volume
      mountPath: /efs-data
  volumes:
  - name: efs-volume
    persistentVolumeClaim:
      claimName: efs-pvc
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$

```

Apply:

kubectl apply -f efs-pod2.yaml

STEP 3 — Verify Pods

kubectl get pods

Both must show:

efs-pod1 Running

efs-pod2 Running

```

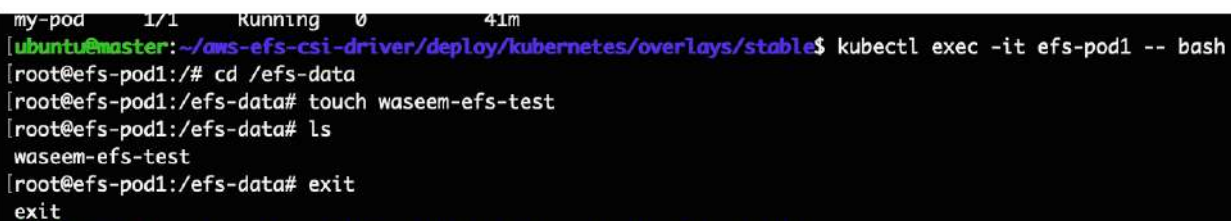
      claimName: efs-pvc
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl apply -f efs-pod2.yaml
pod/efs-pod2 created
ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable$ kubectl get pods
NAME       READY   STATUS    RESTARTS   AGE
efs-pod1   1/1     Running   0           111s
efs-pod2   1/1     Running   0           4s

```

STEP 4 — TESTING

Login to pod1

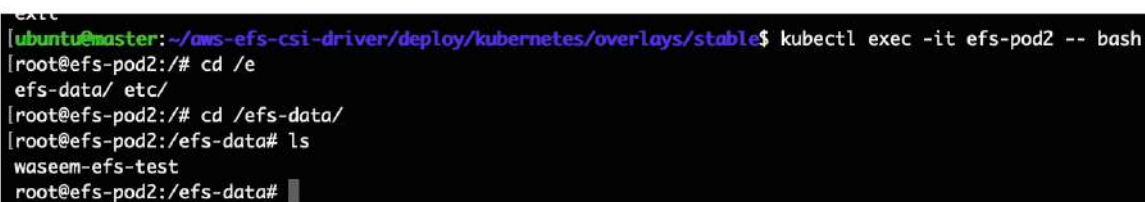
```
kubectl exec -it efs-pod1 -- bash
cd /efs-data
touch waseem-efs-test
ls
exit
```



```
my-pod 1/1 Running 0 41m
[ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable]$ kubectl exec -it efs-pod1 -- bash
[root@efs-pod1:/# cd /efs-data
[root@efs-pod1:/efs-data# touch waseem-efs-test
[root@efs-pod1:/efs-data# ls
waseem-efs-test
[root@efs-pod1:/efs-data# exit
exit
```

Now check from pod2

```
kubectl exec -it efs-pod2 -- ls /efs-data
```



```
exit
[ubuntu@master:~/aws-efs-csi-driver/deploy/kubernetes/overlays/stable]$ kubectl exec -it efs-pod2 -- bash
[root@efs-pod2:/# cd /e
efs-data/ etc/
[root@efs-pod2:/# cd /efs-data/
[root@efs-pod2:/efs-data# ls
waseem-efs-test
root@efs-pod2:/efs-data#
```

You MUST see:

waseem-efs-test

That proves:

- Same volume
- Multiple pods
- ReadWriteMany working
- EFS shared storage success

Task:5

5.Implement and Test Liveness and Readiness Probes in a Kubernetes Pod

Liveness Probe:

=====

Liveness can use with pod configuration to check the health status of pod.
If pod is not healthy for any reason then Liveness will restart the pod.

Why We Need It

Sometimes:

- Application hangs
- Process is stuck
- Memory leak
- Infinite loop
- Port not responding

Even if container is running, app may be dead internally.

Liveness detects this.

PART 1 — Liveness Probe

Step 1: Create YAML file

```
vi liveness.yaml
```

```
ubuntu@master:~$ vi liveness.yaml
[ubuntu@master:~$ cat liveness.yaml
apiVersion: v1
kind: Pod
metadata:
  name: liveness-pod
spec:
  containers:
  - name: my-app
    image: nginx
    ports:
    - containerPort: 80
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 15
      periodSeconds: 10
```

```
ubuntu@master:~$
```

Step 2: Apply YAML

```
kubectl apply -f liveness.yaml
```

```
kubectl get pods
```

Wait until:

```
ubuntu@master:~$ kubectl apply -f liveness.yaml
pod/liveness-pod created
[ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
efs-pod1      1/1     Running   0           15m
efs-pod2      1/1     Running   0           14m
liveness-pod  1/1     Running   0           20s
my-pod        1/1     Running   0           55m
ubuntu@master:~$
```


liveness-pod Running

Step 3: Verify Liveness Probe

kubectl describe pod liveness-pod

```
ubuntu@master:~$ kubectl describe pod liveness-pod
Name:          liveness-pod
Namespace:     default
Priority:       0
Service Account: default
Node:          worker1/172.31.9.88
Start Time:    Wed, 25 Feb 2026 12:21:53 +0000
Labels:        <none>
Annotations:   cni.projectcalico.org/containerID: 79c8db6a8b44426b59d9cd3a3454362cc4ce53c761d3c087950f07792e357f55
               cni.projectcalico.org/podIP: 192.168.235.143/32
               cni.projectcalico.org/podIPs: 192.168.235.143/32
Status:        Running
IP:            192.168.235.143
IPs:           IP: 192.168.235.143
Containers:
  my-app:
    Container ID:   containerd://ff4d2c4731be83169501ab1930ff6cee372fa3750e7ee069ef11effe72c425bd
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3392e
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Wed, 25 Feb 2026 12:21:54 +0000
    Ready:          True
    Restart Count:   0
    Liveness:       http-get http://:80/ delay=15s timeout=1s period=10s #success=1 #failure=3
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-t96p9 (ro)
Conditions:
  Type                     Status
  PodReadyToStartContainers True
  Initialized              True
  Ready                    True
  ContainersReady          True
  PodScheduled             True
Volumes:
  kube-api-access-t96p9:
    Type:                Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:        kube-root-ca.crt
    ConfigMapOptional:    <nil>
    DownwardAPI:          true
QoS Class:               BestEffort
Node-Selectors:           <none>
Tolerations:             node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                         node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason      Age   From          Message
  ----     -
  Normal   Scheduled   112s  default-scheduler  Successfully assigned default/liveness-pod to worker1
  Normal   Pulling     112s  kubelet        Pulling image "nginx"
  Normal   Pulled      112s  kubelet        Successfully pulled image "nginx" in 187ms (187ms including waiting). Image size: 62944796 bytes.
  Normal   Created     112s  kubelet        Created container: my-app
  Normal   Started     111s  kubelet        Started container my-app
ubuntu@master:~$
```

```
Ready:          True
Restart Count:   0
Liveness:       http-get http://:80/ delay=15s timeout=1s period=10s #success=1 #failure=3
Environment:    <none>
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-t96p9 (ro)
```

delay=15s

Kubernetes waits 15 seconds after container starts before checking liveness for the first time.

period=10s

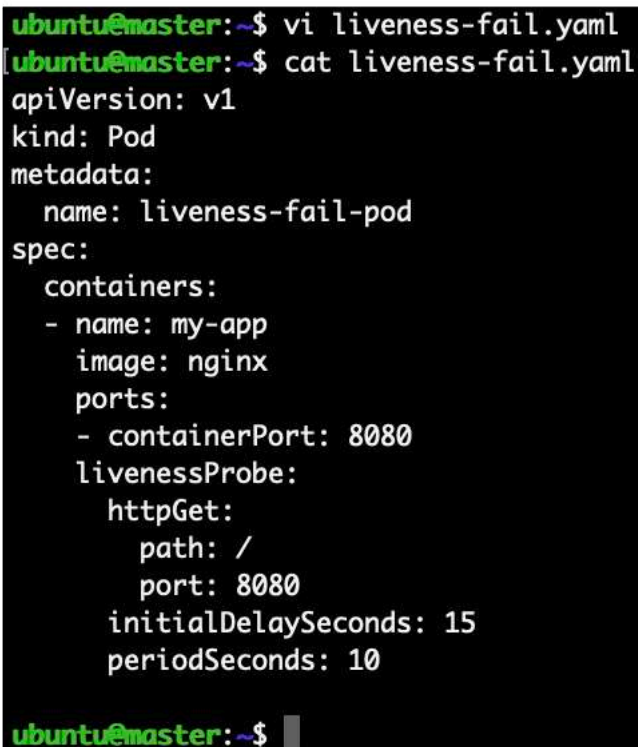
Every 10 seconds kubelet sends HTTP request to:

`http://pod-ip:80/`

Step 4: Test Liveness Failure (Wrong Port)

Create new file:

```
vi liveness-fail.yaml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user enters 'vi liveness-fail.yaml' and then 'cat liveness-fail.yaml'. The output shows a YAML configuration for a Pod. The configuration includes apiVersion: v1, kind: Pod, metadata with name: liveness-fail-pod, and spec with a container named my-app using the nginx image. The container has a port 8080 and a livenessProbe configured with httpGet on path / and port 8080, with initialDelaySeconds: 15 and periodSeconds: 10. The terminal ends with the prompt 'ubuntu@master:~\$' and a cursor.

```
ubuntu@master:~$ vi liveness-fail.yaml
ubuntu@master:~$ cat liveness-fail.yaml
apiVersion: v1
kind: Pod
metadata:
  name: liveness-fail-pod
spec:
  containers:
  - name: my-app
    image: nginx
    ports:
    - containerPort: 8080
    livenessProbe:
      httpGet:
        path: /
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 10
ubuntu@master:~$
```

Apply

```
kubectl apply -f liveness-fail.yaml
```

```
ubuntu@master:~$ kubectl apply -f liveness-fail.yaml
pod/liveness-fail-pod created
ubuntu@master:~$ kubectl get pods -w
NAME                READY   STATUS    RESTARTS   AGE
liveness-fail-pod   1/1     Running   0           7s
liveness-pod        1/1     Running   0          7m16s
```

Step 4 — Watch Pod

```
kubectl get pods -w
```

After about 20–40 seconds you will see:

```
liveness-fail-pod   Running   1/1   RESTARTS 1
```

Then restart count will increase:

```
RESTARTS 2
```

```
RESTARTS 3
```

```
per 10 seconds. 10
ubuntu@master:~$ kubectl apply -f liveness-fail.yaml
pod/liveness-fail-pod created
ubuntu@master:~$ kubectl get pods -w
NAME                READY   STATUS    RESTARTS   AGE
liveness-fail-pod   1/1     Running   0           7s
liveness-pod        1/1     Running   0          7m16s
liveness-fail-pod   1/1     Running   1 (0s ago)  41s
```

Why This Happens

- Nginx actually runs on port 80
- Probe checks port 8080
- No response
- Failure count reaches 3
- Kubernetes restarts container

Step 5 — Confirm Restart

Run:

```
kubectl describe pod liveness-fail-pod
```

```
node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
Type Reason Age From Message
----
Normal Scheduled 2m32s default-scheduler Successfully assigned default/liveness-fail-pod to worker2
Normal Pulled 2m31s kubelet Successfully pulled image "nginx" in 186ms (186ms including waiting). Image size: 62944796 bytes.
Normal Pulled 111s kubelet Successfully pulled image "nginx" in 188ms (188ms including waiting). Image size: 62944796 bytes.
Normal Created 71s (x3 over 2m31s) kubelet Created container: my-app
Normal Started 71s (x3 over 2m31s) kubelet Started container: my-app
Normal Pulled 71s kubelet Successfully pulled image "nginx" in 209ms (209ms including waiting). Image size: 62944796 bytes.
Normal Pulling 31s (x4 over 2m31s) kubelet Pulling image "nginx"
Warning Unhealthy 31s (x9 over 2m11s) kubelet Liveness probe failed: Get "http://192.168.189.75:8080/": dial tcp 192.168.189.75:8080: connect: connection refused
Normal Killing 31s (x3 over 111s) kubelet Container my-app failed liveness probe, will be restarted
ubuntu@master:~$
```

Scroll to Events section.

You will see something like:

Killing container
Container restarted

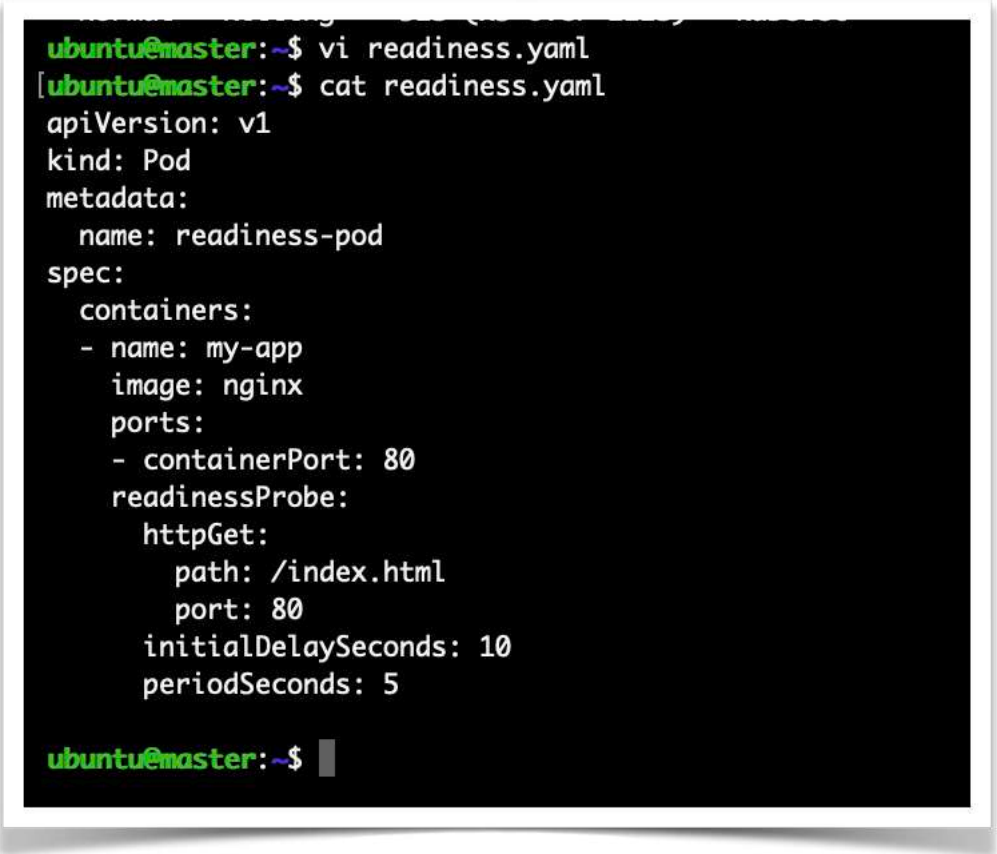
That means:

- ✓ Probe failed
- ✓ Kubernetes detected unhealthy container
- ✓ Kubernetes killed container
- ✓ Container restarted

PART 1 — Create Readiness Pod

Step 1: Create YAML

```
vi readiness.yaml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user enters 'vi readiness.yaml' and then 'cat readiness.yaml'. The output shows a YAML configuration for a Pod named 'readiness-pod' with a container named 'my-app' using the 'nginx' image. It includes a readiness probe configured with an HTTP GET request to '/index.html' on port 80, with an initial delay of 10 seconds and a period of 5 seconds.

```
ubuntu@master:~$ vi readiness.yaml
ubuntu@master:~$ cat readiness.yaml
apiVersion: v1
kind: Pod
metadata:
  name: readiness-pod
spec:
  containers:
  - name: my-app
    image: nginx
    ports:
    - containerPort: 80
    readinessProbe:
      httpGet:
        path: /index.html
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
ubuntu@master:~$
```

Step 2: Apply

```
kubectl apply -f readiness.yaml
```

Step 3: Check Pod

```
kubectl get pods
```

After ~10 seconds you should see:

```
readiness-pod    1/1    Running
```

✓ READY = 1/1

This means readiness probe is passing.

```
ubuntu@master:~$ kubectl apply -f readiness.yaml
pod/readiness-pod created
ubuntu@master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
liveness-fail-pod   0/1     CrashLoopBackOff   6 (110s ago)    7m51s
liveness-pod        1/1     Running           0             15m
readiness-pod       0/1     Running           0             7s
[ubuntu@master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
liveness-fail-pod   0/1     CrashLoopBackOff   6 (2m14s ago)    8m15s
liveness-pod        1/1     Running           0             15m
readiness-pod       1/1     Running           0             31s
ubuntu@master:~$
```

PART 2 — Test Failure

Now we break it.

Step 4: Login to Pod

```
kubectl exec -it readiness-pod -- bash
```

Inside container:

```
cd /usr/share/nginx/html
ls
You will see index.html
```

Delete it:

```
rm index.html
exit
```

```
ubuntu@master:~$ kubectl exec -it readiness-pod -- bash
[root@readiness-pod:/# cd /usr/share/nginx/html
[root@readiness-pod:/usr/share/nginx/html# ls
50x.html index.html
root@readiness-pod:/usr/share/nginx/html# rm index.html
exit
exit
ubuntu@master:~$ kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE
liveness-fail-pod 0/1     CrashLoopBackOff 7 (73s ago) 10m
liveness-pod      1/1     Running             0           17m
readiness-pod     0/1     Running             0           2m50s
```

Step 5: Watch Pod

```
kubectl get pods -w
```

After few seconds you will see:

```
exit
ubuntu@master:~$ kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE
liveness-fail-pod 0/1     CrashLoopBackOff 7 (73s ago) 10m
liveness-pod      1/1     Running             0           17m
readiness-pod     0/1     Running             0           2m50s
```

```
readiness-pod    0/1     Running
```

⚠ Notice:

- Pod is NOT restarted
- Restart count stays 0

- But READY becomes 0/1

Important Difference

Liveness	Readiness
Restarts container	Does NOT restart
Checks if alive	Checks if ready for traffic
If fails → pod restarted	If fails → pod removed from service endpoints

Testing Readiness Probe

1. Create pod with readiness probe.
2. Verify pod status is 1/1 Running.
3. Login into pod.
4. Delete index.html file.
5. Run `kubectl get pods -w`
6. Observe pod becomes 0/1 but not restarted.
7. Restart count remains 0.

This proves:

Readiness probe removes pod from traffic but does not restart container.



What You Have Achieved

Liveness

- Wrong port → Probe failed
- Container restarted
- Restart count increased

Readiness

- Deleted index.html
- Pod became 0/1
- No restart happened

